T13 Context7 & Sequential MCP

Technique Intermediate

Documentation officielle et raisonnement multi-étapes à portée de prompt

Context7 : documentation versionnée à la demande

Context7 résout un problème courant : Claude connaît les APIs populaires jusqu'à sa date de cutoff, mais pas leurs versions récentes ni les patterns officiels exacts. Context7 interroge la documentation officielle d'une bibliothèque et injecte les extraits pertinents directement dans le contexte.

Triggers naturels : présence d'un import statement, question sur une API spécifique, besoin d'un pattern conforme à la doc officielle.

```
# Ce type de prompt déclenche Context7
"Implémente useEffect avec cleanup — React 19"
"Ajoute l'auth Auth0 avec Next.js App Router"
"Configure Prisma pour les transactions imbriquées"
```

Quand privilégier Context7 sur WebSearch : quand la précision de version compte, quand les résultats WebSearch sont trop génériques, ou quand le code doit suivre les patterns officiels sans interprétation.

Sequential Thinking : analyse structurée

Sequential Thinking apporte un moteur de raisonnement multi-étapes pour les problèmes complexes. Plutôt que de répondre directement, il décompose le problème en hypothèses, les teste, et construit une réponse argumentée étape par étape.

Invocation : demandez l'outil MCP Sequential Thinking installé pour un problème qui bénéficie d'étapes explicites. `--think-hard` et `--ultrathink` ne sont pas des flags CLI Claude Code. Le mot `ultrathink` dans un prompt ajoute une instruction de raisonnement sans modifier l'effort API ; utilisez `/effort` pour les modèles compatibles. Haiku 4.5 (historique) n'a pas de paramètre d'effort.

```
# Invoquer Sequential pour un debug complexe
"Pourquoi cette API est-elle lente ? Utilise Sequential Thinking pour
analyser étape par étape."

# Design d'architecture
"Conçois un système de cache multi-niveaux avec Sequential Thinking."
```

Ne pas utiliser pour : corrections simples, typos, explications courtes, tâches à fichier unique. L'overhead de raisonnement ne se justifie pas sur des cas triviaux.

Tableau comparatif

Critère	Context7	Sequential	Claude natif
Usage optimal	Docs bibliothèques	Analyse complexe	Tâches simples
Setup	MCP requis	MCP requis	Aucun
Coût tokens	Moyen	Élevé	Faible
Vitesse	Rapide	Lent (raisonnement)	Très rapide
Précision	Docs officielles	Logique structurée	Connaissance générale

Combinaison efficace

Les deux serveurs se complètent bien pour les tâches d'architecture ou de migration :

1. Context7 fournit les patterns officiels de la bibliothèque cible
2. Sequential analyse comment les intégrer dans le système existant
3. Claude natif exécute les modifications fichier par fichier

```
# Workflow type migration Next.js 14 → 15
1. Context7 → patterns officiels de migration (App Router, caching)
2. Sequential → analyse de l'impact sur le codebase existant
3. Claude natif → éditions des fichiers concernés
```

Installation rapide

```
# Context7 (nécessite une clé API)
npx -y @upstash/context7-mcp --api-key YOUR_API_KEY

# Sequential Thinking (officiel Anthropic)
npx -y @modelcontextprotocol/server-sequential-thinking
```

Enregistrer le serveur avec `claude mcp add --transport stdio --scope project <name> -- npx -y <package>`, en utilisant le paquet choisi et ses arguments requis. Le scope projet écrit `.mcp.json` ; les scopes utilisateur et local résident dans `~/.claude.json`. Inspecter la connexion avec `/mcp`. Voir l'installation MCP.